Software Requirements Specification

IP Information Service (IP2C Integration)

ExpressYourself — Novibet Academy

A REST service that resolves IP addresses to country information, keeps that information fresh through a scheduled job, and reports stored data per country. Built as an integration on an existing clean-architecture skeleton.

Requirements & Design Notes

Three use cases: Get IP Info • Update All IPs • Get Report

Contents

1 Introduction **2**
1.1 Purpose . . . 2
1.2 Scope . . . 2
1.3 External Service: IP2C . . . 2
1.4 Definitions . . . 2

2 Overall Description **2**
2.1 Architecture context . . . 2
2.2 Shared design principles . . . 3
2.3 Assumptions and constraints . . . 3

3 Use Case: Get IP Info (Task 1) **4**
3.1 Purpose & Scope . . . 4
3.2 Functional Requirements . . . 4
3.3 Use Case Detail . . . 4
3.4 Reference Diagram (sequence) . . . 5
3.5 Reference Diagram (activity) . . . 5
3.6 Design Decisions . . . 6
3.7 Open Questions . . . 6

4 Use Case: Update All IPs (Task 2) **7**
4.1 Purpose & Scope . . . 7
4.2 Functional Requirements . . . 7
4.3 Use Case Detail . . . 7
4.4 External Interface Note . . . 8
4.5 Reference Diagram: Overview . . . 8
4.6 Reference Diagram: Process Batch (zoom-in) . . . 8
4.7 Reference Diagram: Save Batch (zoom-in) . . . 9
4.8 Design Decisions . . . 10
4.9 Open Questions . . . 11

5 Use Case: Get Report (Task 3) **12**
5.1 Purpose & Scope . . . 12
5.2 Functional Requirements . . . 12
5.3 Use Case Detail . . . 12
5.4 Data Source . . . 12
5.5 Reference Diagram (activity) . . . 13
5.6 Design Decisions . . . 13
5.7 Open Questions . . . 14

1 Introduction

1.1 Purpose

This document specifies the requirements and records the design decisions for the IP Information Service. The service is the subject of the *ExpressYourself* assignment and is delivered as an integration into an existing clean-architecture solution rather than as a standalone project. The document is organised around three use cases, each treated as a self-contained section following a shared format.

1.2 Scope

The service exposes two HTTP endpoints and one background job:

- **Get IP Info** — returns country information for a single IP address, backed by a cache/database/external-service fallback chain.
- **Update All IPs** — a scheduled job that keeps stored IP information fresh.
- **Get Report** — returns per-country aggregate statistics over stored data.

All three build on the same domain model (`Country`, `Ip`) and the same infrastructure (repository, cache, gateway).

1.3 External Service: IP2C

Country data is sourced from ip2c.org, a free web service polled with one HTTP GET per IP address. Its response is a single semicolon-separated line:

```
status;countryCode2;countryCode3;countryName
```

where `status` is `0` (invalid input), `1` (success), or `2` (unknown IP). On non-success statuses the country fields are not meaningfully populated. The service also accepts a decimal (`?dec=`) request form that is lighter server-side. This contract is shared by all use cases that reach the external service and is not repeated in full within each section.

1.4 Definitions

- **ACL (Anti-Corruption Layer)** — a boundary that translates the external service's format into clean domain objects, so the raw format never propagates inward.
- **CQRS** — separation of read (query) and write (command) paths, dispatched through a mediator.
- **Keyset cursor** — pagination by `WHERE key > lastSeen` rather than offset, giving constant read speed.

2 Overall Description

2.1 Architecture context

The service is integrated into the existing solution and reuses its established patterns: CQRS with a mediator, cross-cutting concerns (caching, logging) applied as decorators, an external gateway acting as an Anti-Corruption Layer, and a scheduled job host. The domain is modelled around two aggregates: `Country` (`TwoLetterCode`, `ThreeLetterCode`, `CountryName`) and `Ip` (`Address`, `CountryTwoLetterCode`).

2.2 Shared design principles

- **Layer separation** — each layer only does what it is allowed to know; the API layer knows HTTP, the application layer orchestrates, the domain holds rules, and infrastructure/gateway handle the outside world.
- **Always-valid domain objects** — a returned `IpInfo` means a valid, successful result; non-success cases are signalled by other means.
- **Integration-minded reuse** — new components mirror existing ones (e.g. the IP2C gateway mirrors the existing ECB gateway) rather than reinventing patterns.

2.3 Assumptions and constraints

- Scope is limited to IPv4. The numeric-key representation (IPv4 $\leftrightarrow$ uint32) does not extend to IPv6.
- No authentication is assumed for the endpoints within this scope.
- Cross-cutting trade-offs that remain open for the code review are collected per use case under *Open Questions*.

3 Use Case: Get IP Info (Task 1)

3.1 Purpose & Scope

Expose an endpoint that returns country information for a given IP address, sourcing the data as cheaply as possible: cache first, then database, then the external IP2C service — persisting and caching any freshly fetched result for later reuse. This section covers only the read path.

3.2 Functional Requirements

- **FR-1** Accept an IPv4 address and return `{ CountryName, TwoLetterCode, ThreeLetterCode }`.
- **FR-2** Resolve in priority order: cache → database → external IP2C service.
- **FR-3** On a cache/db miss, fetch from IP2C, persist to the database, and store in the cache before returning.
- **FR-4** Validate the input as a well-formed IPv4 address before processing.
- **FR-5** Translate the IP2C response into a clean domain object; the raw external format must not propagate beyond the gateway.

3.3 Use Case Detail

Actor: API caller (client) • **Trigger:** `GET /ip/{address}`
Precondition: none (no auth assumed) • **Postcondition:** country info returned; if freshly fetched, now persisted and cached.

Main flow (happy path):

1. Caller sends a dotted IPv4 address.
2. Controller builds the query and dispatches it via the mediator.
3. Handler looks up the cache → hit → return.

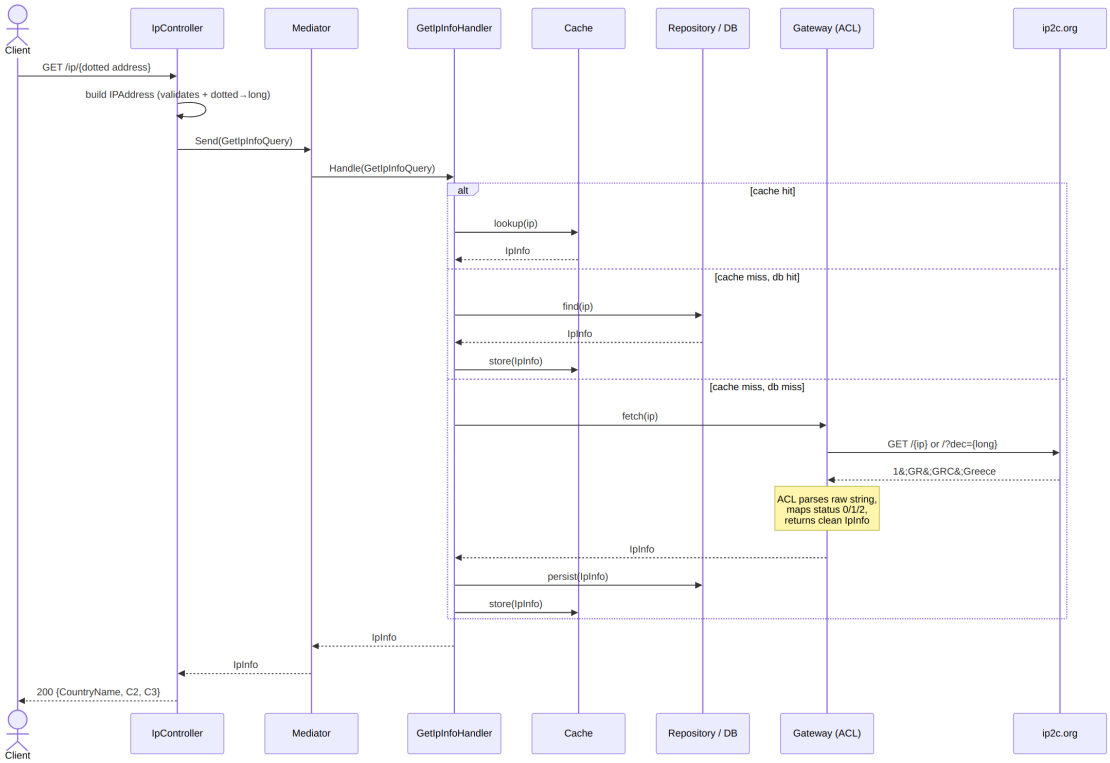
Alternative flows:

- **A1 — cache miss, db hit:** handler queries the database → found → store in cache → return.
- **A2 — cache miss, db miss:** handler calls the gateway → gateway queries IP2C → parses response → returns clean object → handler persists to db + stores in cache → return.

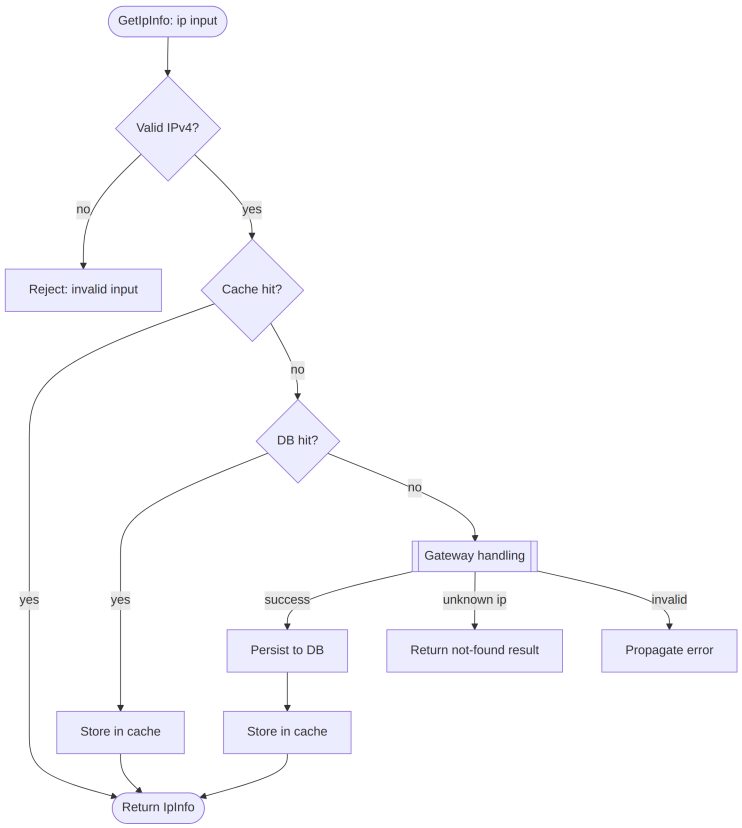
Exception flows (from IP2C `status`):

- **E1 — status 0 (invalid input):** treated as a client error (e.g. `400`).
- **E2 — status 2 (unknown IP):** treated as not-found / empty result (e.g. `404`).
- **E3 — status 1 (success):** proceed with the returned country data.

3.4 Reference Diagram (sequence)



3.5 Reference Diagram (activity)



3.6 Design Decisions

- **CQRS via mediator.** The controller never calls a handler directly; it dispatches a query and the mediator routes it, keeping the API layer ignorant of application internals.
- **Cache-first fallback chain.** Directly serves the performance/cost requirement — external calls are the last resort and their results are reused.
- **Caching as a decorator.** The cache lookup/store wraps the handler rather than living inside it, keeping the handler's logic clean (single responsibility).
- **Gateway as Anti-Corruption Layer.** All IP2C parsing is isolated in the gateway; if IP2C changes format, only the gateway changes. Consistent with the existing `EcbHttpClient` (XML → domain).
- **IP stored as a numeric key (`long`).** The IPv4 ↔ uint32 mapping is bijective, giving a compact, fast, naturally-unique key and trivial dedup. (Boundary: does not extend to IPv6.)

3.7 Open Questions

- **Gateway non-success signalling.** How the gateway reports status 0 and 2 — null, a domain exception, or a `Result` type — is an open trade-off between simplicity and explicitness. Status 0 (invalid) is arguably a bug if input is validated upstream; status 2 (unknown) is an expected outcome, which argues against treating them identically. The existing codebase leans on exceptions.
- **Conversion placement.** Whether the dotted→long conversion lives in the `IPAddress` domain type (triggered at the controller edge) or inside the handler.

4 Use Case: Update All IPs (Task 2)

4.1 Purpose & Scope

Keep stored IP information fresh. On a fixed hourly schedule, the system re-fetches country data from IP2C for the IPs already in the database, updates any that changed, and invalidates their cache entries so stale data is never served. This section covers only the background update path.

4.2 Functional Requirements

- **FR-1** Run a periodic job every one hour.
- **FR-2** Read all stored IPs from the database, processing them in batches (e.g. N at a time) rather than all at once.
- **FR-3** For each IP, call IP2C to fetch the latest country information.
- **FR-4** Update the database only for records whose information has changed, writing the new country data and refreshing their `LastUpdated` timestamp.
- **FR-5** For each changed record, invalidate the corresponding cache entry.
- **FR-6** Be resumable in principle via an ordered cursor over the IP key.

4.3 Use Case Detail

Actor: Scheduler (time-triggered, no HTTP caller) • **Trigger:** hourly timer fires
Precondition: IPs exist in the database • **Postcondition:** every changed IP is updated with a fresh `LastUpdated`, and its stale cache entry is evicted.

Main flow:

1. Job activates; cursor starts at the beginning of the IP key range.
2. Fetch a batch of IPs (ordered by key, N at a time) beyond the cursor.
3. Process the batch: for each record, call IP2C and detect whether the info changed; collect the changed records.
4. Save the batch: persist the changed records in a single transaction and invalidate their cache entries.
5. Advance the cursor to the last key in the batch; repeat from step 2.
6. When a fetch returns an empty batch, the job is complete.

Alternative / edge flows:

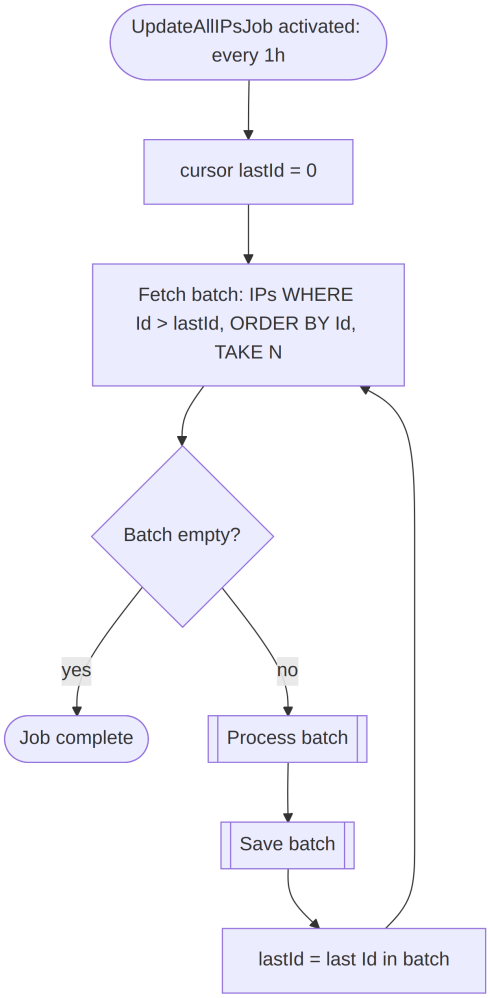
- **A1 — batch has no changes:** nothing is persisted or invalidated; the job advances the cursor and continues.
- **A2 — IP2C returns unknown/invalid for a record:** handled per the gateway's status policy (see Task 1); such a record is not treated as a normal successful update.

4.4 External Interface Note

IP2C is **one call per IP** — there is no bulk endpoint. A batch of N therefore triggers N separate calls; batching applies to the database read (fetch N together) and the database write (save N together), not to the external calls. The `?dec=` request form pairs with the numeric IP key.

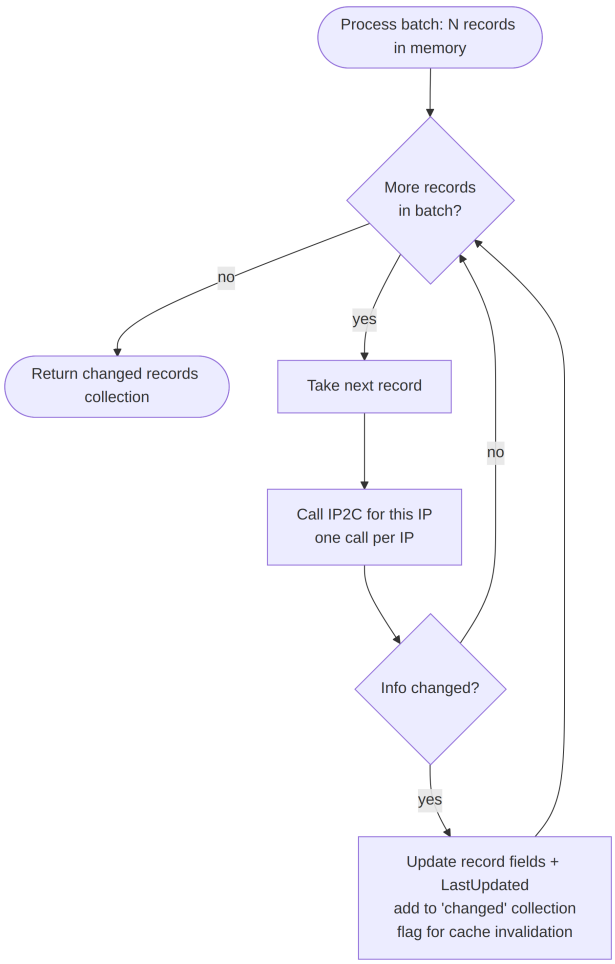
4.5 Reference Diagram: Overview

The job at a glance. Steps with a double border are subprocesses detailed below.



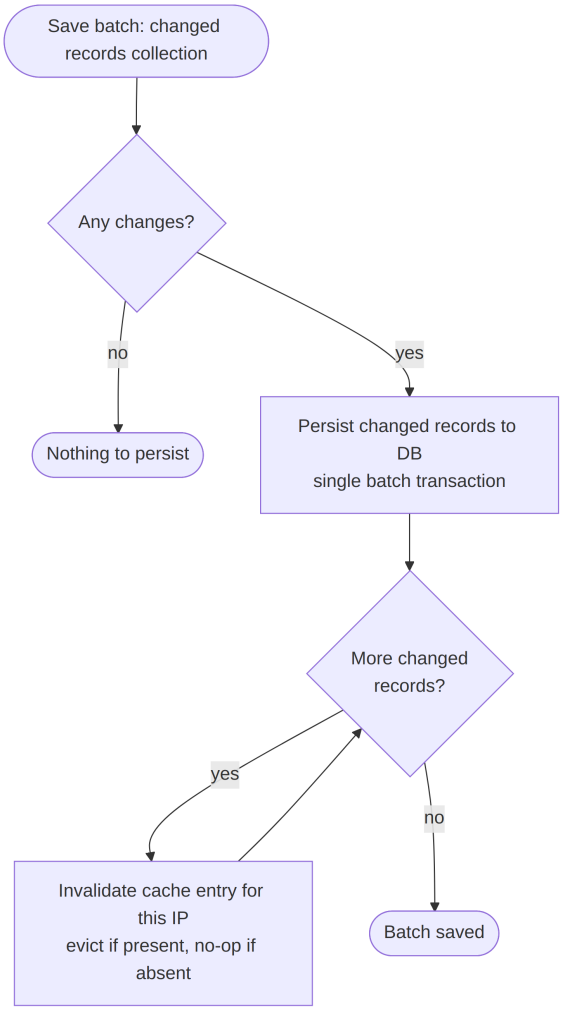
4.6 Reference Diagram: Process Batch (zoom-in)

Reads and decides only; writes nothing. Queries IP2C per record and collects which changed.



4.7 Reference Diagram: Save Batch (zoom-in)

Writes only; decides nothing. Takes the ready collection and persists + invalidates.



4.8 Design Decisions

- **Scheduled background job.** Reuses the existing `DataFetch` job pattern; the job self-triggers on a timer, with no HTTP entry point.
- **Batch processing via keyset cursor.** Fetch `WHERE Id > lastId ORDER BY Id TAKE N`. Constant read speed with an index and naturally resumable, versus offset/skip-take which slows as the offset grows. Pairs with the numeric IP key.
- **Detect/write split.** The batch is processed in two phases — detect changes (read-only, collect) then save (persist + invalidate). Keeps one clean transaction per batch instead of many small writes, and creates the boundary that makes future multithreading feasible.
- **Update over delete+insert.** The IP key is stable; only country fields and `LastUpdated` change, so updating the row avoids identity churn and FK breakage. (Contrast: the WorldRank CurrencyRates job used idempotent delete-then-insert for full-dataset replacement — a different context.)
- **Cache invalidation (evict), not write-through.** Matches the spec's "invalidate the cache"; simpler and safe — eviction is a no-op if the entry is absent. The next read repopulates from the fresh db value.
- **Gateway reuse.** The same Anti-Corruption Layer from Task 1 performs the IP2C call and parsing; the job depends on clean domain objects, never the raw string.

4.9 Open Questions

- **Concurrency.** A single-threaded job is the baseline. Parallelism, if introduced, belongs at the external calls (I/O-bound), bounded by a semaphore to cap concurrent requests, with a producer/consumer split. Whether the added complexity is worth it for an hourly background job is an open decision.
- **Cache analysis.** A deeper analysis of cache behaviour across all three tasks, in the context of the decorator pattern, and across the integrated skeleton, is proposed as its own area.

5 Use Case: Get Report (Task 3)

5.1 Purpose & Scope

Expose an endpoint that reports, per country, how many IP addresses are stored and when an IP of that country was last updated. The caller may request specific countries (by two-letter codes) or all of them. The report reads exclusively from the database; freshness is the responsibility of the update job (Task 2). Keeping "update" and "report" as separate concerns means running update-then-report yields whatever view is needed, without coupling the two.

5.2 Functional Requirements

- **FR-1** Expose an endpoint returning an array of `{ CountryName, AddressesCount, LastAddressUpdated }`.
- **FR-2** Accept an array of two-letter country codes to scope the report, or `null` for all countries.
- **FR-3** For each reported country, count the stored addresses and report the most recent `LastUpdated` among them.
- **FR-4** Validate the input (a set of well-formed two-letter codes, or `null`).
- **FR-5** Read only from the database; do not consult the cache or call IP2C.

5.3 Use Case Detail

Actor: API caller (client) • **Trigger:** `GET /report` (optionally with country codes)
Precondition: none (empty database yields an empty report) • **Postcondition:** an array of per-country rows is returned.

Main flow:

1. Caller sends either a list of two-letter codes or `null`.
2. Input is validated.
3. If codes are provided, the query filters to those countries; if `null`, it includes all.
4. The query groups the stored IPs by country and, per group, counts addresses and takes the most recent `LastUpdated`.
5. The result is shaped into report rows and returned as a JSON array.

Alternative flows:

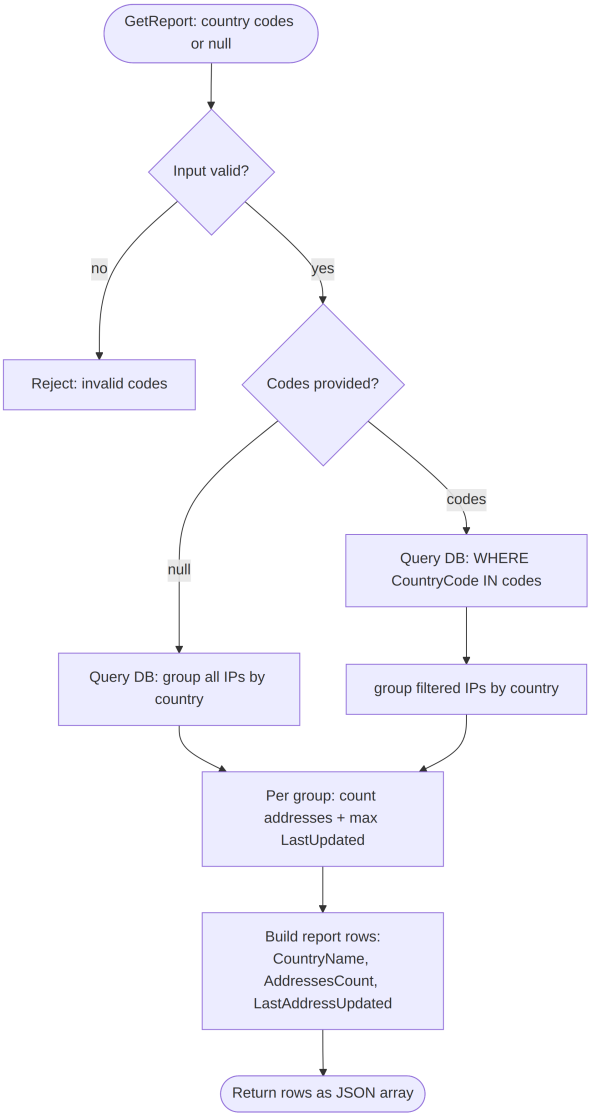
- **A1 — null input:** no filter is applied; all countries are grouped and reported.
- **A2 — codes provided:** the query filters by `CountryCode IN (codes)` before grouping, so only the requested countries are processed.
- **A3 — a requested country has no stored IPs:** it does not appear in the result (or appears with a zero count, depending on join semantics — see Open Questions).

5.4 Data Source

The report reads from the database only. It is an aggregate query over already-stored IP records — a whole-table aggregate, a different access pattern from the single-key lookups the cache serves. No cache lookup, no IP2C call.

5.5 Reference Diagram (activity)

One flow: a conditional filter feeds a shared group + aggregate step.



5.6 Design Decisions

- **Conditional filter, shared aggregate.** The filter (`WHERE CountryCode IN codes`) is applied only when codes are provided; the group + count + max step is written once and reused for both cases. A `null` input skips the filter. Avoids duplicating the aggregate logic across two paths (DRY).
- **Filter-first for scoped reports.** Filtering before grouping lets the database process only the relevant rows (with an index), rather than grouping the whole table and discarding the rest. `WHERE country = X` is just a group-by with a single group.
- **Push the work to the database.** The grouping and aggregation are expressed so EF Core translates them to SQL and executes them in the database, not by loading all rows into memory and grouping in C#. This is the main performance decision.
- **Read-only, db-only source.** Deliberately bypasses the cache and IP2C, keeping "report" cleanly separated from "update".

- **Array-or-null input.** The spec allows a set of codes or `null` — not just a single code. The `IN (codes)` filter covers one, many, or (skipped) all with the same shape.

5.7 Open Questions

- **Missing-country semantics (A3).** If a requested country has no stored IPs, should it be omitted or returned with `AddressesCount = 0`? Omission falls out of an inner grouping; a zero-row requires a left join against a known-countries set. Decide based on caller expectations.